----------Question----------

1. **Public Members (name)**
   No underscore — accessible from anywhere, inside or outside the class.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name      # public
        self.salary = salary  # public

e = Employee("Raj", 50000)
print(e.name)     # Raj   accessible directly
print(e.salary)   # 50000  accessible directly
e.salary = 60000    #  can modify freely too
```

Real-world example: A Product object in an e-commerce app where name and price are meant to be freely read/displayed everywhere on the website.

2. Protected Members (_name)

Single underscore — a convention signaling "internal use, don't touch from outside." Python doesn't actually block access; it's a gentleman's agreement, mainly meant for use within the class and its subclasses.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self._salary = salary   # protected

class Manager(Employee):
    def give_bonus(self, amount):
        self._salary += amount   #  subclass CAN access it
        print(f"New salary: {self._salary}")

m = Manager("Priya", 70000)
m.give_bonus(5000)     # New salary: 75000
print(m._salary)      # 75000   works, but you "shouldn't" do this from outside
```

Real-world example: In a banking system, _interest_rate might be marked protected — subclasses like SavingsAccount or FixedDepositAccount need it to calculate interest, but external code (like a billing UI) shouldn't touch it directly.

3. Private Members (__name)

Double underscore — Python performs name mangling, rewriting __name to _ClassName__name internally. This makes it hard (not impossible) to access from outside the class.

```
class Account:
    def __init__(self, balance):
        self.__balance = balance   # private

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient funds")

    def get_balance(self):
        return self.__balance

acc = Account(1000)
acc.deposit(500)
acc.withdraw(300)
print(acc.get_balance())   # 1200

# print(acc.__balance)     #  AttributeError
print(acc._Account__balance)  # 1200  still accessible via mangled name (not truly private)
```

```
class BankAccount:
    bank_name = "SBI"  # class variable (public)

    def __init__(self, owner, balance, pin):
        self.owner = owner          # public   → shown on statements
        self._branch = "Kanpur"     # protected → used internally/by subclasses
        self.__balance = balance    # private   → sensitive, hidden
        self.__pin = pin            # private   → sensitive, hidden

    def deposit(self, amount):
        self.__balance += amount
        print(f"Deposited {amount}. New balance: {self.__balance}")

    def withdraw(self, amount, pin):
        if pin != self.__pin:
            print("Wrong PIN!")
            return
        if amount > self.__balance:
            print("Insufficient balance")
            return
        self.__balance -= amount
        print(f"Withdrew {amount}. New balance: {self.__balance}")

    def get_balance(self, pin):
        if pin == self.__pin:
            return self.__balance
        return "Access denied"


class SavingsAccount(BankAccount):
```

```
    def add_interest(self, rate):
        interest = self.get_balance(self._pin_check()) if False else None
        # subclass can access _branch (protected) but NOT __balance/__pin directly
        print(f"Branch: {self._branch}")

acc = BankAccount("Amit", 10000, pin=1234)
print(acc.owner)        # Amit        public — freely accessible
print(acc._branch)      # Kanpur      protected — works, but discouraged outside class
# print(acc.__balance)  # AttributeError — private, blocked

acc.deposit(2000)             # Deposited 2000. New balance: 12000
acc.withdraw(500, pin=1234)   # Withdrew 500. New balance: 11500
print(acc.get_balance(1234))  # 11500
print(acc.get_balance(9999))  # Access denied
```

Inheritance

A class (child) can inherit attributes/methods from another class (parent).

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound")

class Cat(Animal):        # single inheritance
    def speak(self):
        print(f"{self.name} says Meow")

class Puppy(Dog, Animal):   # multiple inheritance
    pass
```

Polymorphism

Same interface, different implementations.

```
class Bird:
    def sound(self):
        print("Some bird sound")

class Parrot(Bird):
    def sound(self):
        print("Parrot talks")

class Crow(Bird):
    def sound(self):
        print("Crow caws")

for b in [Parrot(), Crow()]:
    b.sound()   # method overriding + polymorphism
```

Constructors and Destructors

```
class Demo:
    def __init__(self):       # constructor
        print("Object created")
    def __del__(self):        # destructor
        print("Object destroyed")
```

. Class Variables vs Instance Variables----------

```
class Employee:
    company = "TCS"         # class variable — shared by all
```

```
    def __init__(self, name):
        self.name = name      # instance variable — unique per object
```

Method Overriding vs Overloading

- **Overriding: child class redefines a parent method (shown above with Bird)**
- **Overloading: Python doesn't support true overloading, but you can mimic it with default args:**

```
class Calc:
    def add(self, a, b=0, c=0):
        return a + b + c

print(Calc().add(2, 3))      # 5
print(Calc().add(2, 3, 4))   # 9
```

super()

Used to call the parent class's methods/constructor.

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand)
        self.model = model

c = Car("Toyota", "Corolla")
print(c.brand, c.model)
```

How Python Runs Your Code (Simple Version)

Think of it like a translation chain — your code passes through several stages before the computer actually runs it.

1. You write code

python

x = 5 + 3

2. Python breaks it into pieces (Tokenizing)
It splits your code into small parts like: x, =, 5, +, 3 — just like breaking a sentence into words.

3. Python understands the structure (Parsing → AST)
It figures out what these words *mean together* — "oh, this is an assignment: put the result of 5+3 into x." This forms a tree-like structure in memory.

4. Python converts it into bytecode (Compiling)
This tree is turned into simple step-by-step instructions (like a recipe) that the computer's Python engine can follow. This is saved in a .pyc file so it doesn't have to redo this next time.

5. The Python engine runs it (PVM)
The Python Virtual Machine reads these instructions one by one and actually performs them — like a chef following the recipe steps.

Code → Words → Tree (meaning) → Instructions → Engine runs it → Result

Step	What Happens
Write code	You type Python
Tokenize	Split into pieces
Parse	Understand structure
Compile	Turn into instructions
Run (PVM)	Engine executes instructions
GIL	Only one thing runs at a time
Frames	Track each function call

Main Types of Memory

1. Stack Memory

Used for function calls and local variables. Fast, automatically managed, cleared when a function ends.

```
def add(a, b):
    result = a + b   # 'a', 'b', 'result' live in stack memory (frame)
    return result
```

- **Stores: function calls, local variables, references**
- **Cleared automatically when function returns**
- **Very fast, limited size**

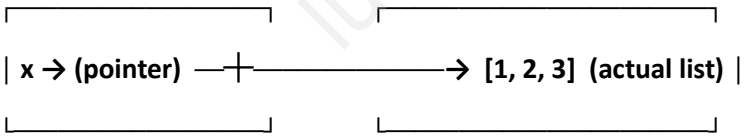
Heap Memory

Used for actual objects — lists, dicts, class instances, strings, etc. This is where Python's reference counting + garbage collector work.

```
x = [1, 2, 3]   # the list itself lives in heap memory
                # 'x' (in stack) just points to it
```

- **Stores: all Python objects (int, str, list, dict, custom classes)**
- **Managed by Python's memory manager (pymalloc) + GC**
- **Slower than stack, but flexible size**

Stack (fast, temporary) Heap (slower, long-lived)



Code/Text Memory

Stores the actual compiled bytecode of your program (the instructions).

Static/Global Memory

Stores global variables and things that live for the entire program's life.

```
counter = 0   # global — lives as long as the program runs
```

Memory Type	Stores	Speed	Managed by
Stack	Function calls, local vars	Very fast	Automatic (LIFO)
Heap	Objects (list, dict, class, etc.)	Slower	Refcounting + GC
Code	Compiled bytecode	—	Interpreter
Global/Static	Global variables	—	Lives whole program

Write a program to check if a number is prime.

2. Write a program to reverse a string without using slicing ([::-1]) or built-in reverse functions.

3. Write a program to check if a string is a palindrome.

4. Write a program to find the factorial of a number using both a loop and recursion.

5. Write a program that counts the frequency of each character in a string.

6. Write a program to find the largest and smallest number in a list without using min()/max().

7. Write a program to remove duplicates from a list while preserving order.

8. Write a function that takes a list of numbers and returns a new list containing only the even numbers, using list comprehension.

9. Write a program to check if two strings are anagrams of each other.

10. Implement a function using *args and **kwargs that accepts any number of positional and keyword arguments and prints them.

11. Write a program to find the second largest number in a list in a single pass (one loop, no sorting).

12. Create a custom exception class InsufficientBalanceError and use it in a simple bank withdrawal function.

13. Write a generator function that yields Fibonacci numbers up to n terms.

Write a decorator @timer that prints how long a function took to execute.

15. Implement a Stack class using a list, with push, pop, peek, and is_empty methods.